CIÊNCIA DA COMPUTAÇÃO · UNISINOS · COMPILADORES

Trabalho de Grau A (TGA): analisador léxico e gramática de declarações

Enunciado oficial, requisitos de implementação, critérios de avaliação e cronograma de acompanhamento.

Trabalho	Analisador léxico e gramática de declarações de uma linguagem de programação em estilo C. Corresponde a 50% da nota do Grau A; os demais 50% provêm da prova do grau.
Equipe	Individual ou em dupla.
Ferramenta	Flex (recomendada) ou, alternativamente, PLY (Python Lex-Yacc) ou Lark, em Python. A rubrica de avaliação é a mesma nas três opções. Todas são gratuitas e executam no Google Colab, sem instalação local.
Entrega	No Moodle, na tarefa correspondente, até as 19h30 do dia da apresentação. A data exata consta na própria tarefa.
Apresentação	Presencial, no dia da entrega, com aproximadamente 10 minutos por trabalho e demonstração ao vivo.

1. Objetivo

Especificar e implementar as duas primeiras camadas de um tradutor para uma linguagem de programação em estilo C: a gramática da parte de declarações e o analisador léxico completo, executável e com tratamento de erros. São essas as camadas sobre as quais qualquer tradutor é construído, e o exercício reproduz, em escala reduzida, o mesmo percurso adotado na construção de compiladores reais.

Ao final, a dupla deverá ser capaz de justificar cada decisão de especificação, demonstrar o analisador em execução sobre programas de teste e explicar o comportamento do código entregue, linha a linha.

2. A linguagem a ser reconhecida

A linguagem desta etapa contempla apenas a parte de declarações. As regras que a definem são as seguintes:

- Três tipos são reconhecidos: **char**, **int** e **float**.
- Identificadores seguem o padrão da linguagem C: iniciam por letra ou sublinhado e, do segundo caractere em diante, aceitam letras, dígitos e sublinhado.
- As declarações podem ser individuais ou em lista separada por vírgulas, como em **int a, b, c;**, e terminam sempre em ponto e vírgula.
- Nesta etapa as variáveis são apenas declaradas, sem inicialização.
- Espaços, tabulações e quebras de linha são ignorados. Qualquer outro caractere constitui erro léxico, que deve ser reportado com o número da linha.

Programas válidos e inválidos

Programas válidos	Programas inválidos

`int a;`
`float x, y, z;`
`char nome;`
`int _contador, total2;`
	`int 2contador;`
	`float x = 3.5;`
	`double z;`
	`int a b;`

O escopo da linguagem é intencionalmente restrito, de modo que o mecanismo de reconhecimento seja dominado integralmente, em lugar de abranger um número elevado de construções.

3. Requisitos de implementação

- **Especificação escrita dos tokens**, com as respectivas expressões regulares, e da gramática de declarações, em arquivo editável, com as decisões documentadas.
- **Analisador léxico executável**: arquivo .l do Flex ou script em Python, que imprima a sequência de tokens reconhecidos e a tabela de símbolos.
- **Programas de teste**: no mínimo três válidos e três com erros, léxicos e de estrutura, com as saídas comentadas no relatório.
- **Mensagens de erro com o número da linha**, para todo erro detectado.
- **Relatório no template da disciplina**, contendo especificação, decisões de projeto, testes e análise dos resultados.

4. Entregáveis

A entrega é feita pelo Moodle, na tarefa correspondente, até as 19h30 do dia da apresentação, e deve conter:

1. **Especificação escrita**: os tokens com as respectivas expressões regulares e a gramática de declarações, em arquivo editável, com as decisões justificadas.
2. **Analisador executável**: arquivo .l do Flex ou script em Python, em execução no Google Colab sem ajustes, com nomes claros nos objetos e variáveis.
3. **Programas de teste**: no mínimo três válidos e três inválidos, com as saídas comentadas.
4. **Relatório** no template da disciplina, disponível no mesmo tópico do Moodle.

5. Critérios de avaliação

Critério	O que se espera	Peso
Especificação dos tokens e da gramática	Correta, completa e bem documentada, com as decisões justificadas.	3,0
Implementação executável	Reconhece os casos válidos, acusa os inválidos com a linha e executa no Google Colab sem ajustes.	4,0
Testes e análise	Casos bem escolhidos, inclusive de canto, e discussão honesta dos resultados.	2,0
Apresentação	Demonstração ao vivo de aproximadamente 10 minutos, com vídeo de reserva.	1,0

Em síntese, 70% da nota corresponde à engenharia do trabalho, isto é, à especificação e à implementação, aspectos que devem concentrar o maior esforço.

6. Acompanhamento

O trabalho é acompanhado por cinco checkpoints, realizados dentro das próprias aulas, nos últimos minutos do encontro. O checkpoint não compõe a nota, mas sua dispensa tende a acarretar atraso no cronograma do trabalho. As datas de cada encontro seguem o calendário da turma, publicado no Moodle.

Momento	O que deve estar pronto
Checkpoint 1 · no encontro de proposição	Duplas formadas e expressões regulares dos tokens rascunhadas.
Checkpoint 2 · no encontro seguinte	Tokens fechados contra a especificação.
Checkpoint 3 · adiante	Analisador léxico executável em funcionamento no Google Colab.
Checkpoint 4 · mais adiante	Gramática de declarações revisada.
Checkpoint 5 · antes da entrega	Lista de verificação da entrega e ensaio da apresentação.

7. Apresentação

A apresentação ocorre presencialmente no dia da entrega e dura aproximadamente dez minutos por trabalho. Espera-se uma demonstração ao vivo do analisador em execução, seguida da leitura dos resultados sobre os programas de teste e de perguntas da turma e do professor. Recomenda-se a gravação prévia de um vídeo de reserva da demonstração, como precaução contra falhas de equipamento.

8. Erros frequentes

Três equívocos são recorrentes em especificações léxicas iniciais e devem ser observados desde o princípio:

- **Identificador que aceita dígito no início.** O padrão [a-zA-Z0-9_]+ está incorreto, pois aceita 2contador. O correto separa o primeiro caractere: [a-zA-Z_][a-zA-Z0-9_]*.
- **Ponto não escapado.** Em [0-9]+.[0-9]+ o ponto significa "qualquer caractere". Para o ponto literal, escreva \..
- **Palavra reservada absorvida pelo identificador.** A cadeia int casa também com o padrão de identificador; a ordem das regras resolve o desempate, com as reservadas declaradas antes do identificador.

9. Regras e observações

- **Uso de inteligência artificial (IA)**: permitido como assistente, com declaração no relatório. Cada linha entregue precisa ser explicada na apresentação.
- **Erros**: toda mensagem de erro deve informar o número da linha em que ocorreu.
- **Entrega**: no Moodle, até as 19h30 do dia da apresentação. Envios posteriores são registrados como atraso.

10. Materiais de apoio

- Flex, gerador de analisadores léxicos: manual oficial em westes.github.io/flex/manual
- PLY (Python Lex-Yacc), alternativa em Python: ply.readthedocs.io
- Lark, alternativa em Python: lark-parser.readthedocs.io
- Teste de expressões regulares no navegador: regex101.com
- Execução sem instalação: Google Colab, colab.research.google.com

Avisos, orientações complementares e entregas ficam no Moodle da disciplina.